

Lab02_Caparrós_ME315A

Enzo Caparrós Carvalho

Verificando Dados

```
library(dplyr)
library(readr)
library(ggplot2)
library(ggcal)
```

Hands-On

Exercício 1:

Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (*ARRIVAL_DELAY* > 10)?

R: As estatísticas suficientes são o número total de voos e o número de voos que apresentaram atraso na chegada superior a 10 minutos. Com elas é possível calcular o percentual de voos atrasados dividindo o número de voos com *ARRIVAL_DELAY* > 10 pelo número total de voos, para cada combinação de dia, mês e companhia aérea.

Exercício 2:

Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do pacote `dplyr`):

- Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: **AA, DL, UA e US**;
- Remova observações que tenham valores faltantes em campos de interesse;
- Agrupe o conjunto de dados resultante de acordo com: **dia, mês e cia. aérea**;

- d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe **tibble**;
- e. A função deve receber apenas dois argumentos:
 - i. **input**: o conjunto de dados (referente ao lote em questão);
 - ii. **pos**: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos) {  
  
  input |>  
  filter(AIRLINE %in% c("AA", "DL", "UA", "US")) |>  
  filter(!is.na(ARRIVAL_DELAY)) |>  
  group_by(DAY, MONTH, AIRLINE) |>  
  summarise(  
    total = n(),  
    atrasados = sum(ARRIVAL_DELAY > 10),  
    .groups = "drop"  
  )  
}
```

Exercício 3:

Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

- a. Configure o tamanho do lote (chunk) para **100 mil registros**;
- b. Configure a função de *callback* para instanciar DataFrames aplicando a função `getStats` criada acima;
- c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as **colunas de interesse** (veja nota de aula para identificar como realizar esta tarefa).

```
callback <- DataFrameCallback$new(getStats)  
  
stats_lotes <- read_csv_chunked(  
  "flights.csv.zip",  
  callback = callback,  
  chunk_size = 100000,  
  col_types = cols_only(  
    DAY = col_integer(),  
    MONTH = col_integer(),
```

```

    AIRLINE = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

```

Exercício 4:

Crie uma função chamada `computeStats` que:

- a. Combine as **estatísticas suficientes** para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- b. Retorne as informações em um **tibble** contendo apenas as seguintes colunas:
 - i. **Cia**: sigla da companhia aérea;
 - ii. **Data**: data, no formato **AAAA-MM-DD** (dica: utilize o comando `as.Date`);
 - iii. **Perc**: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo **[0,1]**.

```

computeStats <- function(stats) {

  stats |>
    group_by(DAY, MONTH, AIRLINE) |>
    summarise(
      total = sum(total),
      atrasados = sum(atrasados),
      .groups = "drop"
    ) |>
    mutate(
      Cia = AIRLINE,
      Data = as.Date(
        paste(2015, MONTH, DAY, sep = "-")
      ),
      Perc = atrasados / total
    ) |>
    select(Cia, Data, Perc)
}

```

Exercício 5:

Exiba as linhas da tabela `PRESERVE` em que `STATE >= 'M'`.

```

stats <- computeStats(stats_lotes)
pal <- scale_fill_gradient(
  low = "#4575b4",
  high = "#d73027"
)

baseCalendario <- function(stats, cia) {

  dados <- stats |>
    filter(Cia == cia)

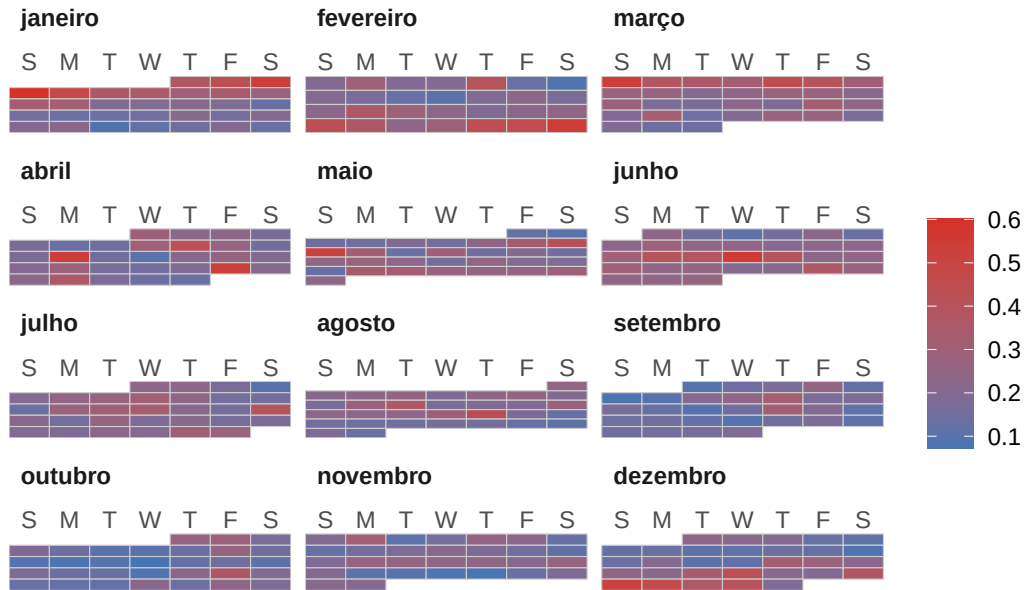
  ggcal(
    dados$Data,
    dados$Perc
  )
}

cAA <- baseCalendario(stats, "AA")
cDL <- baseCalendario(stats, "DL")
cUA <- baseCalendario(stats, "UA")
cUS <- baseCalendario(stats, "US")

cAA + pal + ggtitle("American Airlines (AA)")

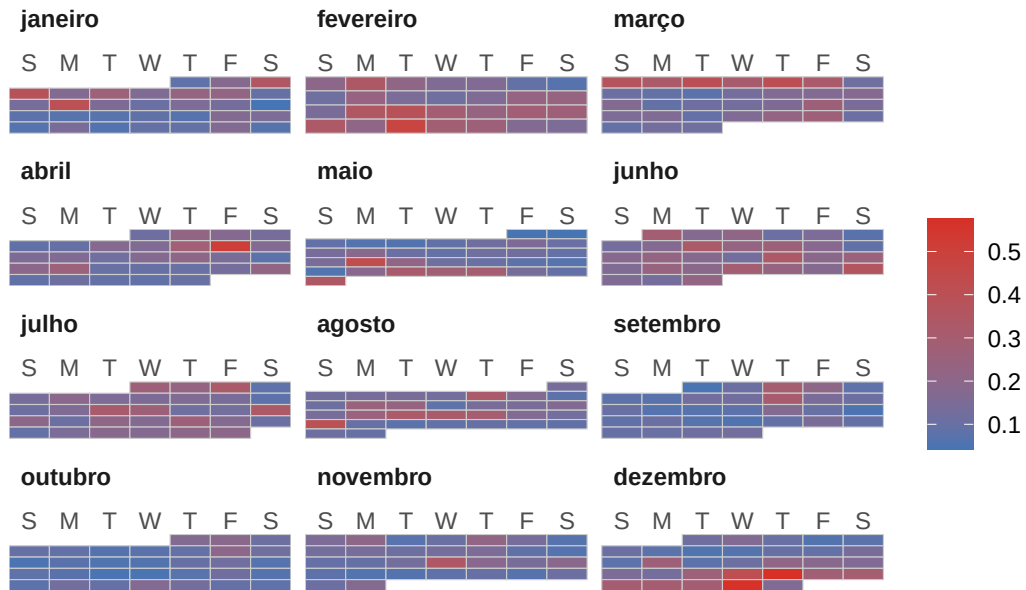
```

American Airlines (AA)



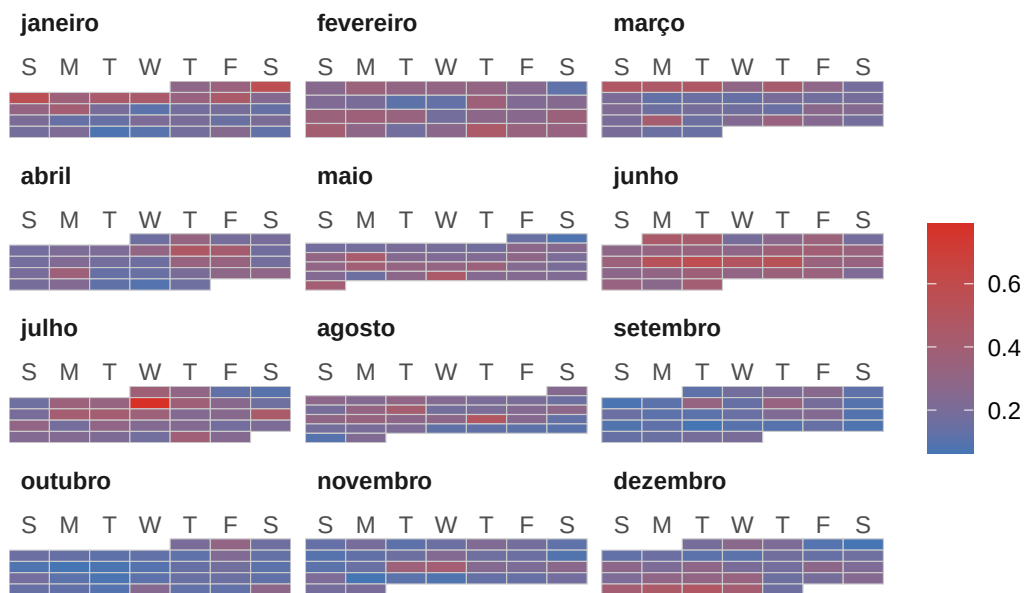
```
cDL + pal + ggtitle("Delta Air Lines (DL)")
```

Delta Air Lines (DL)



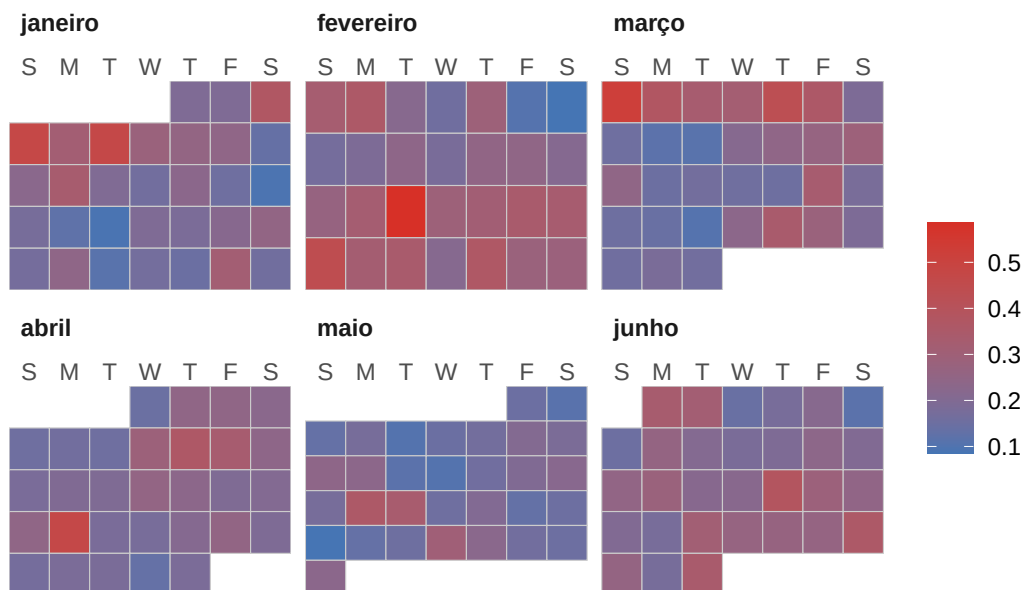
```
cUA + pal + ggtitle("United Airlines (UA)")
```

United Airlines (UA)



```
cUS + pal + ggtitle("US Airways (US)")
```

US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 21:18:28 -03"
```